Discriminative Project

Milestone 1

Study Group 9

1. Introduction

This project looks at a simple question with a practical answer. We are given photographs of 73 different objects, and we want a model that can look at one photo and tell us which object it is. Each object has its own folder of about a hundred pictures, taken from different angles and against different backgrounds, so the model has to recognise the object itself and not just the setting it happens to be sitting in.

We tried three models and compared them. The first is a convolutional neural network that we built and trained from scratch, which means it starts out knowing nothing about images at all. The other two, ResNet-18 and Swin-Tiny, were first trained by other researchers on ImageNet, a very large collection of labelled photos, and we reused that knowledge and adjusted it for our 73 objects. The point of the comparison is to find out how much that head start is worth when we only have a small number of pictures for each object. We measured each model on accuracy, on how well it does class by class, and on its size and speed, so that the choice between them is based on more than one number.

2. Background and Approach

Training an image model from scratch means the network has to learn everything from the pixels up: first simple things like edges and colours, then textures, then shapes, and only after all of that the actual objects. That takes a lot of examples. With only about a hundred pictures per object, there is not much to learn from, and a from-scratch model can struggle.

Transfer learning is a way around this. Instead of starting blank, we take a network that has already been trained on a very large dataset (ImageNet, which has over a million photos) and reuse it. Such a network has already learned good general features, so we only have to teach it the new set of objects. We do this by replacing its last layer with a fresh one that outputs 73 scores, and then training, letting the older layers change only slightly. We chose three models on purpose to cover a range: a small convolutional network we built ourselves as a baseline, ResNet-18 as a classic and efficient pretrained convolutional network, and Swin-Tiny as a modern transformer-based model. Comparing them tells us both whether pretraining helps and whether the newer transformer design is worth its extra size here.

3. The Dataset

The dataset has 7,343 images in total, organised into 73 folders named images_OBJ001 through images_OBJ073, one folder per object. On average each folder holds about a hundred images. The

photos of each object are taken from several angles and on different surfaces, so the same object looks fairly different from one picture to the next. A small number of the images, 379 in total, are pictures of the background on its own with no object in them. We kept these on purpose, because they teach the model what an empty scene looks like and make it less likely to name an object just from its usual background.

Figure 1 shows one example from twelve of the classes, so you can get a feel for the kind of photos we are working with. The objects are ordinary items photographed close up. Figure 2 shows how many images each class has. The dataset is fairly balanced: most classes sit around a hundred images, with a few having slightly more, and none of them is badly under-represented, which means we do not have to worry much about the model ignoring rare classes.

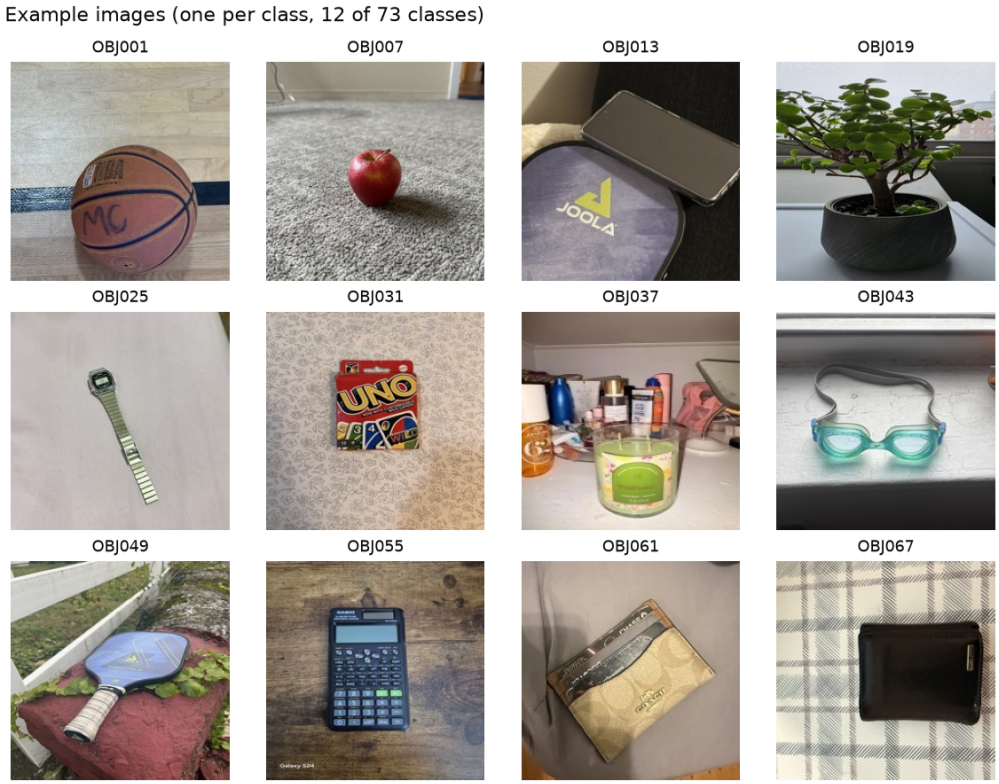


Figure 1. One example image from twelve of the 73 classes.

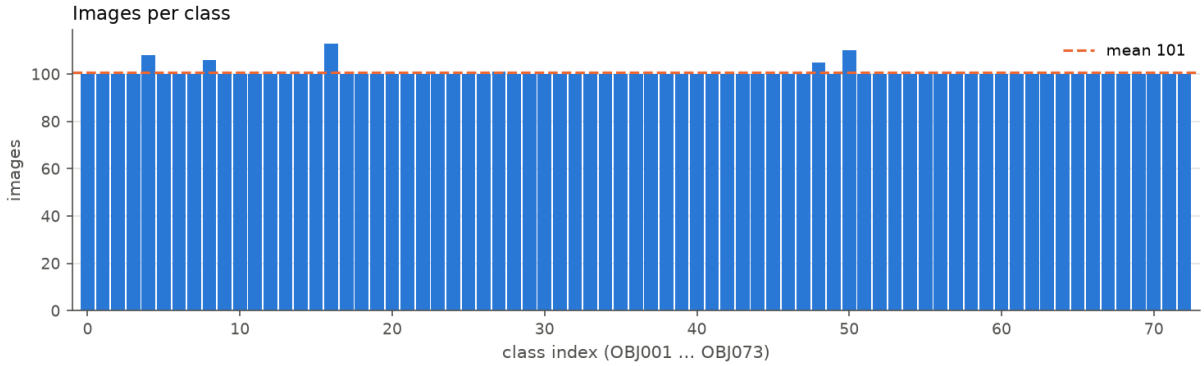


Figure 2. Number of images in each class. The dashed line is the average. The classes are close to balanced.

4. Preparing the Images

Before an image goes into a model we fix its orientation, because phones often store photos rotated, convert it to colour, and resize it to 224 by 224 pixels, which is the size all three models expect. We use the same stratified split for every model so the comparison is fair: 70 percent of the images for training, 15 percent for validation and 15 percent for testing, chosen with a fixed random seed. That works out to 5,140 training images, 1,101 for validation and 1,102 for testing. Using the same split everywhere means any difference we see comes from the model and not from a lucky division of the data.

During training we change each image a little every time it is used. We take a random crop and resize it, sometimes flip it left to right, rotate it by a small angle, and shift the brightness, contrast and colour slightly. This is called data augmentation, and it shows the model many small variations of the same picture so it copes better with new photos. For validation and testing we do none of this. We simply resize the image and pass it in as it is, so the scores reflect ordinary pictures and not augmented ones. We also scale the pixel values in a standard way. The two pretrained models use the same averages that were used when they were first trained on ImageNet, while the from-scratch model uses the averages measured from our own training images.

5. The Models

All three models end in a layer that gives a score for each of the 73 classes, and the highest score is the prediction. What differs is how much they already knew before we started and how they are built inside.

5.1 Custom CNN, trained from scratch

This is a plain convolutional network that we trained from zero. It has five stages. Each stage runs a convolution, normalises the result, applies a ReLU activation and then halves the image size with max pooling. The number of filters starts at 32 and doubles at each stage. At the end the features are averaged into a single vector, passed through a dropout layer that helps prevent overfitting, and finally a linear layer produces the 73 scores. It has about 1.6 million parameters, which is small compared with the other two.

5.2 ResNet-18, transfer learning

ResNet-18 is a well known residual network with about 11 million parameters. We loaded the version already trained on ImageNet, replaced its final layer with a new one sized for 73 classes, and then carried on training. The new layer learns at the normal rate, while the rest of the network is adjusted far more gently, at a tenth of that rate, so the useful features it already has are not wiped out.

5.3 Swin-Tiny, transfer learning

Swin-Tiny is a vision transformer, a newer kind of network that uses attention rather than ordinary convolutions. It is the largest of the three, with about 27.6 million parameters. We set it up in the same way as ResNet-18: load the ImageNet weights, swap the final layer for a 73-class one, and fine-tune

the whole thing gently.

6. How We Trained Them

All three models were trained in the same way so that the comparison stays fair. We used the AdamW optimiser with a cosine schedule that slowly lowers the learning rate as training goes on. The loss was cross-entropy with a small amount of label smoothing, which stops the model from getting too confident about any single answer. After every epoch we checked accuracy on the validation set and kept the version of the model from the best epoch, rather than just the last one. The main training settings are listed in Table 1.

Setting	Value
Optimiser	AdamW
Learning rate	0.001 for the new head, 0.0001 for a pretrained backbone
Schedule	Cosine annealing
Loss	Cross-entropy with 0.1 label smoothing
Batch size	32
Image size	224 x 224
Split	70% train / 15% validation / 15% test (fixed seed)
Epochs	Custom CNN 40, ResNet-18 15, Swin-Tiny 8

Table 1. Training settings, shared by all three models.

We report two accuracy numbers. The first is plain top-1 accuracy, the share of test images named correctly. The second is the macro F1 score, which averages the score across classes so that a few weak classes cannot be hidden by the strong ones. We also measured how long each model takes to classify a single image on the CPU, measured the same way for all three, so the speed numbers can be compared directly.

7. Results

Model	Val acc	Test acc	Test F1	Params (M)	Latency (ms)	Best epoch
Swin-Tiny	**0.962**	**0.948**	**0.949**	**27.6**	**29.1**	**7**
ResNet-18	0.945	0.936	0.936	11.2	10.3	14
Custom CNN	0.799	0.789	0.788	1.6	7.8	39

Table 2. Results on the held-out test set, ordered by validation accuracy (best row shaded). Latency is the time to classify one image on the CPU.

Table 2 is the main result of the project. The two pretrained models are well ahead of the one we trained from scratch. Swin-Tiny is the most accurate at 94.8 percent on the test set, ResNet-18 is close behind at 93.6 percent, and the custom CNN reaches 78.9 percent. That is a gap of about 16

points between the from-scratch model and the pretrained ones. The F1 scores are almost identical to the plain accuracy for each model, which is a good sign: it means the models are not just doing well on the common classes while quietly failing on others, but are spread fairly evenly across all 73. Figure 3 shows the same accuracy numbers as bars.

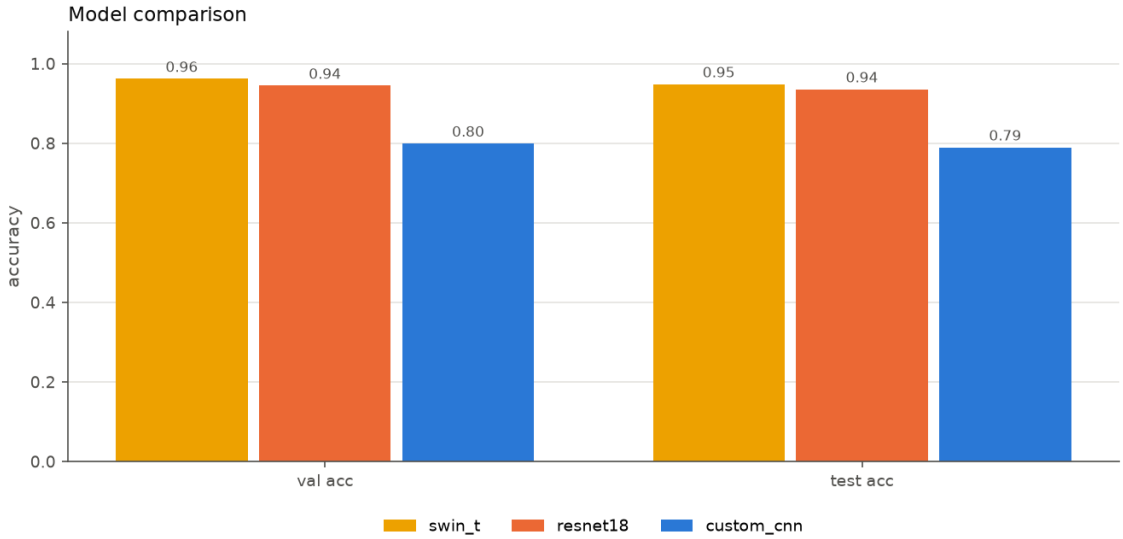


Figure 3. Validation and test accuracy for each model.

8. How the Models Learned

Figure 4 shows what happened during training. The solid lines are validation scores and the dashed lines are training scores. The two pretrained models shoot up to high accuracy within the first few epochs and then flatten out, which is why they did not need long training runs. The custom CNN climbs slowly across all 40 of its epochs and settles at a lower level. The dashed training lines sit a little above the solid validation lines, which is normal and shows a small, healthy amount of overfitting rather than a real problem. The loss curves on the right tell the same story from the other direction, falling fast for the pretrained models and more slowly for the one built from scratch.

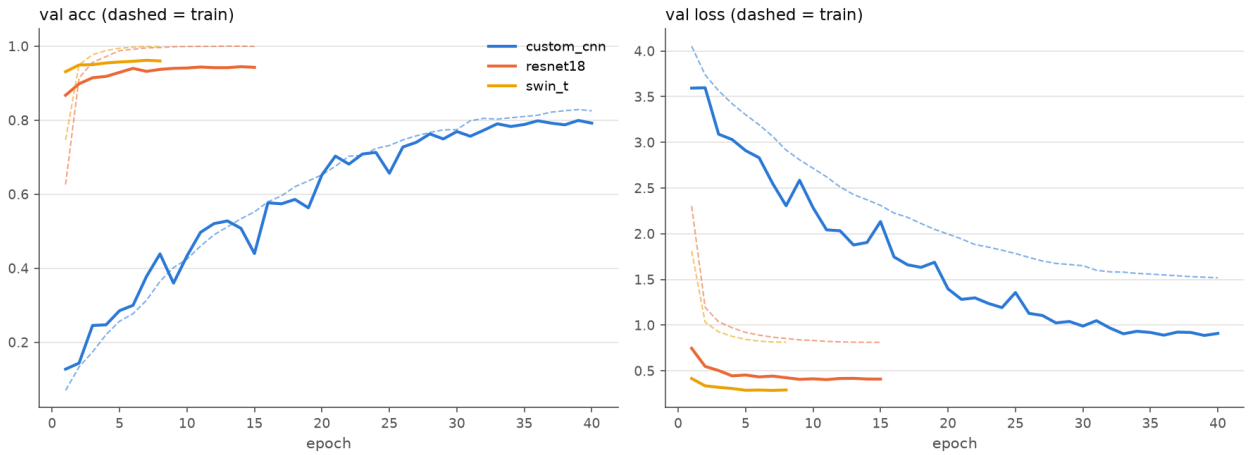


Figure 4. Training (dashed) and validation (solid) accuracy and loss per epoch.

9. Validation Against Test

One worry with any model is that it might look good on the data used to pick it but do worse on data it has never seen. Figure 5 puts each model's validation and test accuracy side by side. For all three the test score is only a little below the validation score, so the models are not simply memorising, and the numbers should hold up reasonably well on fresh photos of the same objects. This small, steady gap is what we want to see, and it gives us some confidence that the ranking in Table 2 is real and not an accident of how the data was split.

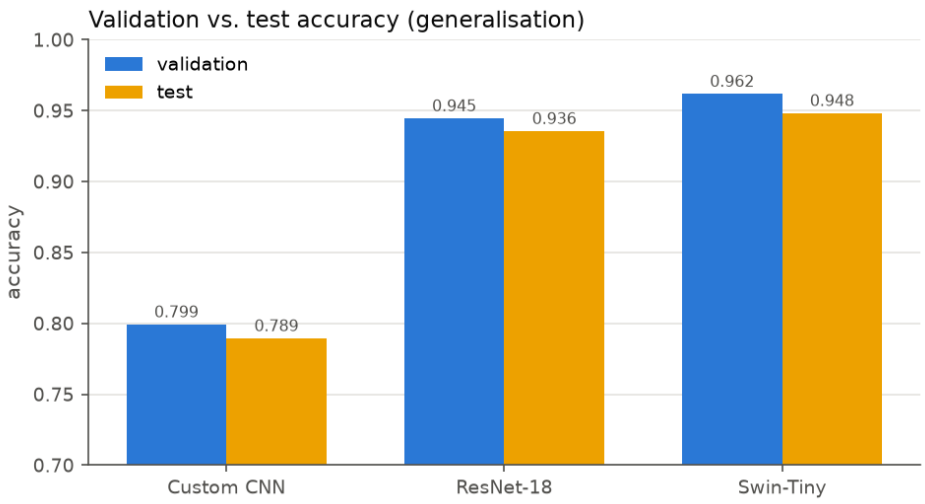


Figure 5. Validation versus test accuracy for each model.

10. Accuracy Against Cost

Accuracy is not the only thing that matters. A model also has a size and a running cost, and in practice you often have to trade one against the other. Figure 6 plots test accuracy against the number of parameters on the left, and against the time to process one image on the right. Swin-Tiny is the most accurate, but it is also the biggest model and the slowest to run, at about 29 milliseconds per image. ResNet-18 gives up only about one point of accuracy while being roughly two and a half times smaller and about three times faster. The custom CNN is the smallest and quickest of all, but its accuracy is not high enough to recommend it here.

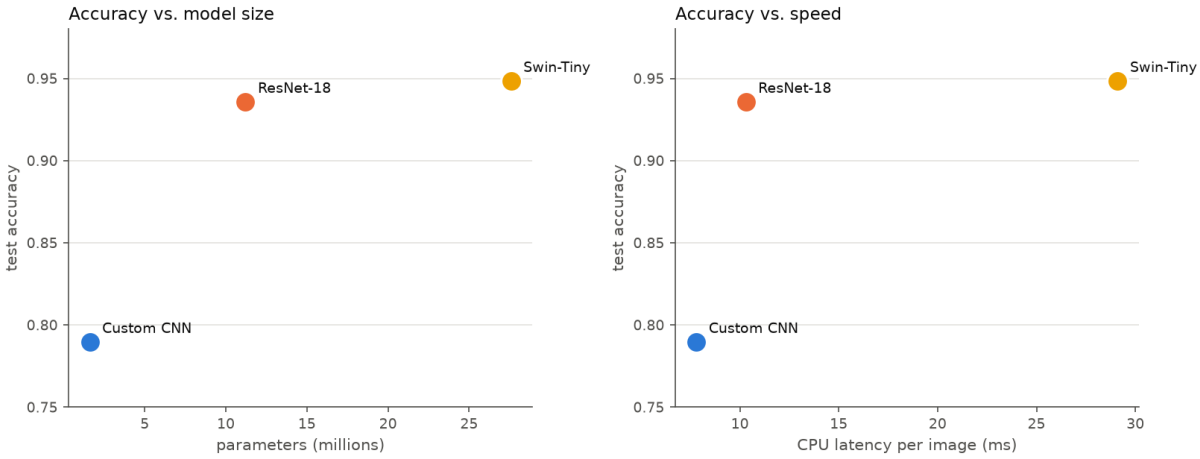


Figure 6. Test accuracy against model size (left) and against speed (right).

Figure 7 shows how long each model took to train. These numbers should be read loosely, because the models were trained on different hardware and one at a time, so they are not measured under identical conditions. They still give a rough sense of the effort involved, and they line up with intuition: the from-scratch model, which has to learn everything, takes the most training despite being the smallest.

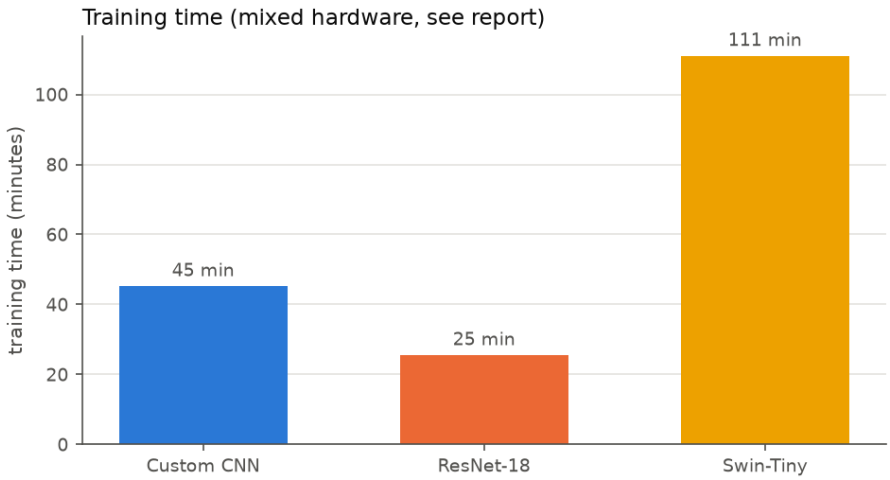


Figure 7. Approximate training time for each model.

11. Where the Models Make Mistakes

To see where the models go wrong we looked at the confusion matrices in Figure 8. Each grid is 73 by 73. The rows are the true object and the columns are the model's guess, so a perfect model would put all of its weight on the diagonal. For Swin-Tiny and ResNet-18 the grids are almost completely diagonal, which means their mistakes are few and scattered. The custom CNN, on the right, has noticeably more off-diagonal spots, so it mixes up more pairs of objects.

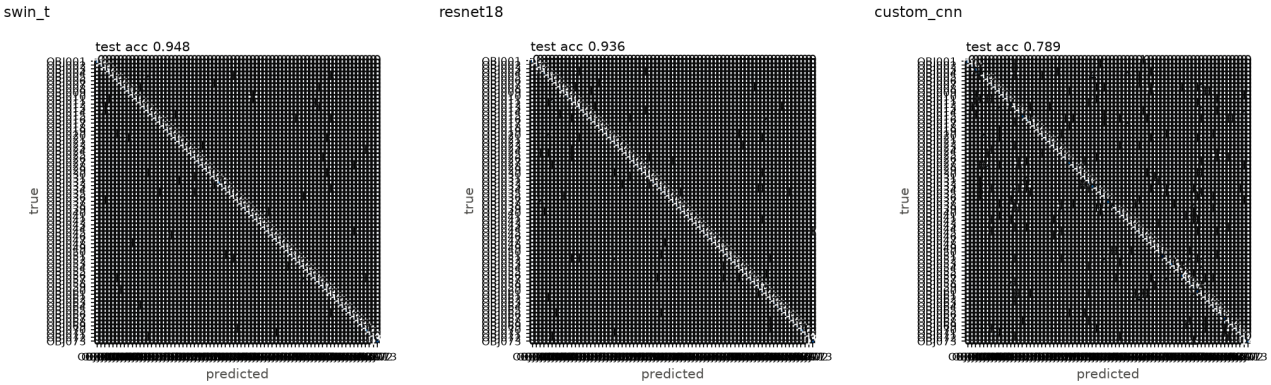


Figure 8. Confusion matrices on the test set: Swin-Tiny, ResNet-18 and Custom CNN (left to right). Darker on the diagonal means correct.

Figure 9 takes a closer look at the best model, Swin-Tiny. The left panel shows how its accuracy is spread across the 73 classes. Most classes are at or near 100 percent and only a few are lower. The right panel lists the classes it found hardest. Its weakest were OBJ057 and OBJ005, at about 73 and 75 percent. By comparison the custom CNN's weakest class, OBJ034, scored only 27 percent, which again shows how much steadier the pretrained models are across the board.

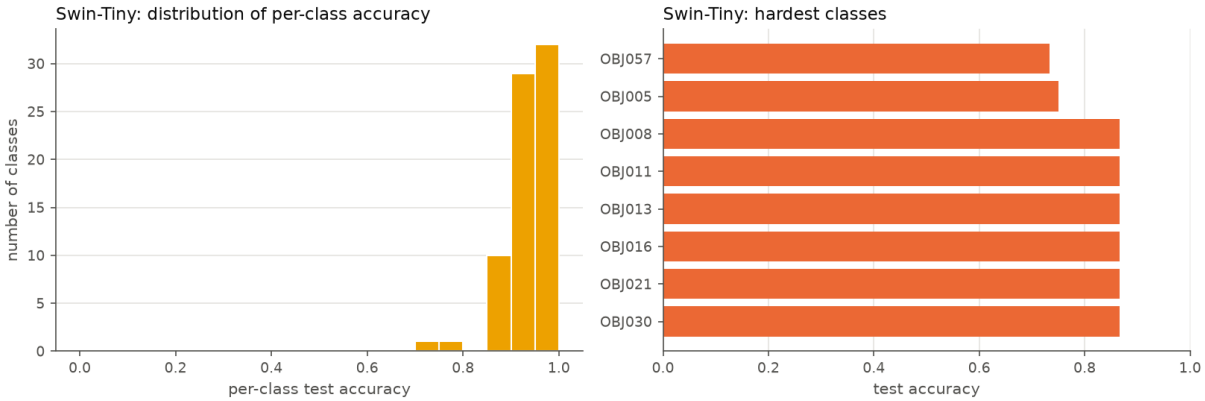


Figure 9. Swin-Tiny: spread of per-class accuracy (left) and its hardest classes (right).

Table 3 lists the object pairs that each model confused most often. The mistakes are usually between objects that look similar, and there are not many of them. Even the most common confusion happens only a few times across the whole test set, which has about fifteen images per class. This is the kind of error we would expect: the models slip on genuinely similar-looking items rather than making random guesses, and the few objects that are hard for one model tend to be hard for the others too.

Model	True object	Guessed as	Times
Swin-Tiny	OBJ059	OBJ061	2
	OBJ057	OBJ054	2
	OBJ005	OBJ014	2
ResNet-18	OBJ059	OBJ061	2
	OBJ037	OBJ003	2

	OBJ052	OBJ022	2
Custom CNN	OBJ062	OBJ045	4
	OBJ005	OBJ033	4
	OBJ068	OBJ067	3

Table 3. The three most common confusions for each model.

12. Discussion

The main lesson from this project is clear. When you only have about a hundred pictures for each class, starting from a model that already understands images in general beats building one from nothing. ResNet-18 and Swin-Tiny spent most of their training just adjusting features they already had, which is why they learned quickly and ended up accurate. The custom CNN had to learn everything at once, from basic edges and textures up to the objects themselves, out of a small dataset, and it simply could not reach the same level in the time available.

Choosing between the two pretrained models comes down to what you care about. If accuracy is the only thing that matters, Swin-Tiny is the best choice. If you also care about speed, memory, or running the model on a modest device, ResNet-18 is the better pick, since it is almost as accurate while being smaller and faster. The custom CNN still earns its place as a baseline. It shows how far a plain network gets on its own and makes the value of pretraining easy to see. It is also worth remembering that the transformer model, Swin-Tiny, did come out on top, but only by about a point over the older ResNet design, so the newer architecture is not a huge jump for a task of this size.

13. Notes on the Setup

A few honest notes about how the runs were done. Because of limits on the computer we used, the models were trained on a mix of the GPU and the CPU, and only one at a time. For that reason the training-time numbers in Table 2 and Figure 7 were not all measured under the same conditions, and they should be read as rough guidance rather than exact figures. The accuracy, F1 and single-image latency numbers, on the other hand, were all measured in a consistent way and can be compared directly. We also used different numbers of epochs for the three models, 40 for the custom CNN and far fewer for the two pretrained ones. This is reasonable, because the training curves show the pretrained models stop improving after only a few epochs while the from-scratch model keeps improving for much longer, so giving them all 40 epochs would have wasted time without changing the result.

14. What We Would Try Next

There are a few things we did not have time for that would likely help. Training the two pretrained models for a few more epochs, or unfreezing their backbones a little more, might push their accuracy up slightly, though the curves suggest the gains would be small. For the custom CNN, a deeper network or more aggressive augmentation might close some of the gap, but it is unlikely to catch the pretrained models given the amount of data we have. Collecting more images for the handful of hard

classes, such as OBJ005 and OBJ057, would probably be the most effective single change, since most of the remaining errors are concentrated in a small number of similar-looking objects.

15. Reproducing This Work

Everything in this report can be reproduced from the code in the repository. The main file, class_identification.py, holds all three models together with the data loading, the training loop, the evaluation and the plotting. The repository also includes a short README with setup steps, a requirements file that pins the exact library versions we used, and a results folder with the figures shown here.

To run it, you create a Python environment, install the requirements, place the dataset folder next to the script, and run a single command to train and compare the models. The code chooses the fastest device available on its own, and on the first run it downloads the pretrained weights for ResNet-18 and Swin-Tiny. The same command rebuilds the comparison table and the figures in this report, so the numbers we report can be checked from scratch. There are also separate commands for searching the custom network's settings and for classifying a single new image with the best saved model.

16. Conclusion

We compared three models on a 73-class object identification task. Transfer learning from ImageNet was the clear winner. Swin-Tiny reached 94.8 percent accuracy on the test set and ResNet-18 reached 93.6 percent, while a convolutional network trained from scratch reached 78.9 percent. For everyday use we would choose ResNet-18, because it is nearly as accurate as Swin-Tiny while being smaller and faster, and we would keep Swin-Tiny for cases where accuracy is the only concern. All of the code and the figures in this report can be reproduced from the repository linked below.

Code repository: https://github.com/Sneha0504/ie7615-object-identification